

Digital Banking Platform: Code Structure Guide

This document provides a recommended code structure for the Digital Banking Platform. It is designed to maximize code reusability between the Web (ReactJS) and Mobile (React Native) applications, in line with the principles outlined in the architectural strategy.

Monorepo Hierarchy

We will use a monorepo organized into three primary top-level directories: apps, packages, and features.

```
digital-banking-platform/
├── apps/
│   ├── web-host/      # Web Host Application (ReactJS Shell)
│   └── mobile-host/   # Mobile Host Application (React Native Shell)
├── packages/
│   ├── platform-core/ # SHARED: The Shared Platform Module (Logic)
│   └── design-system/  # SHARED: The Design System (UI)
└── features/
    └── feature-transfers/ # EXAMPLE: A Micro-Frontend Feature
```

1. packages/: The Shared Foundation

This directory contains modules that are shared across all applications and features. It is the implementation of the **Shared Platform Module**.

packages/platform-core

This package contains all shared, non-UI business logic. It has no dependency on React DOM or React Native.

```
packages/platform-core/
├── src/
│   ├── api/
│   │   └── bankingClient.js # Centralized, secure client for all backend API calls
│   ├── auth/
│   │   └── useAuthSession.js # Hook for managing user authentication state and tokens
│   ├── state/
│   │   └── userProfileStore.js # Global state management (e.g., Zustand or Redux store)
```

```

└─ utils/
  └─ formatters.js    # e.g., formatCurrency(), formatDate()
  └─ validators.js    # e.g., validateAccountNumber()

```

packages/design-system

This package contains the visual foundation. It separates the shared theme from the platform-specific UI components.

```

packages/design-system/
└─ src/
  └─ theme/
    └─ index.js        # Shared design tokens (colors, fonts, spacing)
  └─ web/
    └─ Button.jsx      # WEB: Renders an HTML <button>
    └─ Input.jsx       # WEB: Renders an HTML <input>
  └─ native/
    └─ Button.jsx      # NATIVE: Renders a React Native <Button>
    └─ Input.jsx       # NATIVE: Renders a React Native <TextInput>

```

2. apps/: The Host Applications

These are the lean "shell" applications that users install or visit. Their primary job is to host and orchestrate the remote features.

```

apps/
└─ web-host/
  └─ src/
    └─ App.jsx         # Handles routing, layout, and loading remote Web Components
    └─ webpack.config.js # Module Federation config to consume remotes
  └─ mobile-host/
    └─ src/
      └─ App.jsx       # Handles native navigation and loading remote RN components
      └─ webpack.config.js # Re.Pack config for Module Federation

```

3. features/: Reusable Business Capabilities

This directory contains the individual micro-frontends. Each feature folder is an independent project that can be developed and deployed separately.

Example: features/feature-transfers

This example shows how a "Funds Transfer" feature is structured to maximize code sharing.

features/feature-transfers/

```
|— src/
|   |— **hooks/**
|   |   |— **useTransfers.js** # <<<<<<<<<< THE SHARED LOGIC >>>>>>>>>>>>>>
|   |       # This custom hook contains all business logic:
|   |       # - Fetches accounts via `platform-core/api`
|   |       # - Manages form state (amount, recipient)
|   |       # - Handles form validation using `platform-core/utils`
|   |       # - Submits the transfer request
|   |
|   |— **web/**
|   |   |— TransfersView.jsx # WEB VIEW: Renders HTML form elements (<input>, <button>)
|   |       # - Imports and uses the shared `useTransfers` hook
|   |       # - Uses components from `design-system/web`
|   |   |— index.js          # Exposes the React component as a Web Component
|   |
|   |— **native/**
|   |   |— TransfersView.jsx # NATIVE VIEW: Renders React Native components (<TextInput>,
|   |   |   <Button>)
|   |       # - Imports and uses the EXACT SAME `useTransfers` hook
|   |       # - Uses components from `design-system/native`
|   |   |— index.js          # Exposes the React Native component for the host app
|   |
|   |— webpack.config.js      # Module Federation config to expose the web/native modules
|   |— package.json
```

By structuring the code this way, the `useTransfers.js` hook—which contains all the complex business rules and API interactions—is written **only once**. The platform-specific `TransfersView.jsx` files are simply thin, presentational layers that consume this shared logic, perfectly achieving the "Build Once, Deploy Everywhere" principle.